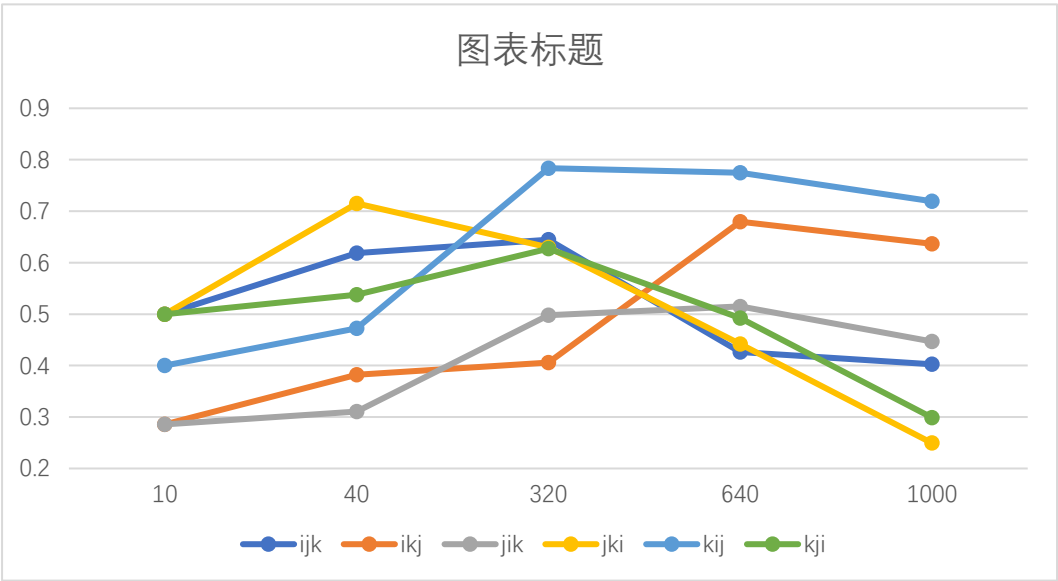


Lab9-实验 1

naive_dgemm 的单核优化

1. 调整 ikj 顺序:

	10	40	320	640	1000
ijk	0.5	0.618357	0.644621	0.426059	0.402565
ikj	0.285714	0.38209	0.405517	0.679632	0.636828
jik	0.285714	0.31068	0.49807	0.514996	0.446598
jki	0.5	0.715084	0.629924	0.442116	0.249552
kij	0.4	0.472325	0.783633	0.774937	0.71931
kji	0.5	0.537815	0.627265	0.492194	0.299041



- 1) ikj, kji 和 jki 在较小矩阵尺寸 (如 10 和 40) 时表现不错, 但随着矩阵尺寸增大 (如 320、640、1000), 性能下降明显。其中 jki 在小矩阵尺寸表现最佳。
- 2) ikj 和 kij 小矩阵 (10、40) 表现不佳, 在 320、640 和 1000 的大矩阵上表现优异。其中 kij 在大矩阵尺寸表现最好。

2. 矩阵分块

	ijk	矩阵分块后（分块大小64）
320	0.644621	0.74098
640	0.426059	0.640248
1000	0.402565	0.595828

可以发现，对于大矩阵尺寸，矩阵分块后性能提升明显。并且随矩阵尺寸增加，提升幅度逐渐下降。

矩阵分块代码：

```
// 分块乘法
for (int ii = 0; ii < m; ii += block_size) {
    for (int jj = 0; jj < n; jj += block_size) {
        for (int kk = 0; kk < k; kk += block_size) {
            for (int i = ii; i < ii + block_size && i < m; i++) {
                for (int j = jj; j < jj + block_size && j < n; j++) {
                    double sum = C[i][j];
                    for (int p = kk; p < kk + block_size && p < k; p++) {
                        sum += A[i][p] * B[p][j];
                    }
                    C[i][j] = sum;
                }
            }
        }
    }
}
```

3. 矩阵分块后重排

	ijk	矩阵分块后（分块大小64）	矩阵分块后重排
320	0.644621	0.74098	0.603991
640	0.426059	0.640248	0.71822
1000	0.402565	0.595828	0.672063

可以发现，相比于分块，对于大矩阵尺寸，重排后性能继续提升。

重排函数

```
for (int ii = 0; ii < rows; ii += block_size) {  
    for (int jj = 0; jj < cols; jj += block_size) {  
        for (int i = ii; i < ii + block_size && i < rows; i++) {  
            for (int j = jj; j < jj + block_size && j < cols; j++) {  
                reordered_matrix[i][j] = orig_matrix[i][j];  
            }  
        }  
    }  
}
```